

Stack Data Structure - Complete Question Bank with Explanations

Question 1: Stack Underflow

Question: In a stack, if a user tries to remove an element from an empty stack it is called?

Answer: Underflow

Explanation: Underflow occurs when you attempt to pop or access elements from an empty stack. This is an error condition because there are no elements to remove. It's the opposite of overflow, which happens when you try to add elements to a full stack.

Question 2: Stack Overflow

Question: Pushing an element into stack already having five elements and stack size of 5, then stack becomes?

Answer: Overflow

Explanation: When a stack has reached its maximum capacity (5 elements in this case) and you try to push another element, it results in an overflow condition. This typically happens in fixed-size array implementations of stacks.

Question 3: Stack Order

Question: Which order is used by stack data structure?

Answer: LIFO (Last In First Out)

Explanation: Stack follows the LIFO principle - the last element added to the stack is the first one to be removed. Think of it like a stack of plates: you add plates on top and remove from the top as well.

Question 4: Stack Declaration in C++ STL

Question: Which of the following is the correct way to declare a stack in C++ using STL?

Answer: `stack<int> myStack;`

Explanation: The correct syntax uses lowercase `stack`, template angle brackets with the data type, semicolon at the end, and no parentheses. The STL stack is declared in the `<stack>` header file.

Question 5: Time Complexity of Push in Linked List Stack

Question: What is the time complexity of the push() operation in a stack implemented as a linked list?

Answer: $O(1)$

Explanation: Pushing an element onto a stack implemented with a linked list takes constant time because you simply create a new node and update the top pointer. No traversal is needed.

Question 6: Removing from Bottom of Stack

Question: In a stack, which operation removes an element from the bottom of the stack?

Answer: None

Explanation: By definition, stacks only allow removal from the top. There is no standard stack operation to remove elements from the bottom. If you need to access the bottom, you'd need to pop all elements above it first, which violates the stack principle.

Question 7: Purpose of empty() Function

Question: What is the purpose of the empty() function in a C++ stack?

Answer: Checks if the stack is empty

Explanation: The `empty()` function returns `true` if the stack has no elements and `false` otherwise. It's commonly used in loops to process all stack elements or to check before performing pop operations.

Question 8: Largest Area in Histogram - Time Complexity

Question: What is the time complexity of the optimal algorithm to find the largest area in a histogram using a stack?

Answer: $O(n)$

Explanation: Using a stack-based approach, we can find the largest rectangular area in a histogram in linear time. Each bar is pushed and popped from the stack at most once, resulting in $O(n)$ complexity.

Question 9: Histogram Algorithm Approach

Question: How does the algorithm for finding the largest area in a histogram using a stack work?

Answer: It iterates through the histogram bars and maintains a stack of indices.

Explanation: The algorithm maintains a stack of bar indices in increasing order of heights. When a bar with smaller height is encountered, it pops bars from the stack and calculates areas with the popped bar as the smallest bar.

Question 10: Largest Histogram Area Calculation

Question: For heights = [6, 2, 5, 4, 5, 1, 6], what is the area of the largest rectangle?

Answer: 12

Explanation: The largest rectangle has:

- Height = 4 (considering bars at indices 2, 3, 4)
- Width = 3 bars
- Area = $4 \times 3 = 12$

This is formed by taking the minimum height in a contiguous sequence and multiplying by its width.

Question 11: Stack Operations Sequence

Question: Output for: push(5), push(8), pop, push(2), push(5), pop, pop, pop, push(1), pop

Answer: 82551

Explanation:

- push(5) → Stack: [5]
- push(8) → Stack: [5,8]
- pop → Output: **8**, Stack: [5]
- push(2) → Stack: [5,2]
- push(5) → Stack: [5,2,5]
- pop → Output: **5**, Stack: [5,2]
- pop → Output: **2**, Stack: [5]
- pop → Output: **5**, Stack: []
- push(1) → Stack: [1]
- pop → Output: **1**, Stack: []

Reading outputs: 8, 2, 5, 5, 1 → **82551**

Question 12: Stack Header File

Question: In C++, which header file is used for stack operations?

Answer: `<stack>`

Explanation: The C++ STL provides the stack container adapter in the `<stack>` header file. You must include this to use the stack data structure.

Question 13: Container Adapters for Stack

Question: In C++, which container adapter can be used to implement a stack?

Answer: All of the above (Vector, Deque, Linked List)

Explanation: The C++ STL stack is a container adapter that can use different underlying containers: `vector`, `deque` (default), or `list`. You can specify: `stack<int, vector<int>> s;`

Question 14: Pop Operation Time Complexity (Array)

Question: What is the time complexity of `pop()` operation in a stack implemented using an array?

Answer: $O(1)$

Explanation: Popping from an array-based stack simply decrements the top index and optionally retrieves the value. No shifting of elements is needed, making it constant time.

Question 15: Empty Operation Time Complexity (Linked List)

Question: What is the time complexity of `empty()` operation in a stack implemented using a linked list?

Answer: $O(1)$

Explanation: Checking if a stack is empty only requires checking if the top pointer is null, which is a constant-time operation regardless of implementation.

Question 16: Stack Output - LIFO Order

Question: Output of pushing 10, 30, 20 and then popping all?

```
cpp
```

```
stack<int> s;
s.push(10);
s.push(30);
s.push(20);
while(!s.empty()){
    cout << s.top() << " ";
    s.pop();
}
```

Answer: 20 30 10

Explanation: Stack follows LIFO order:

- Push: 10 → [10]
 - Push: 30 → [10, 30]
 - Push: 20 → [10, 30, 20]
 - Pop sequence: 20, 30, 10
-

Question 17: Stack Operations Output

Question: Output of size, top, pop, top, empty operations?

```
cpp

s.push(10);
s.push(20);
s.push(30);
cout << s.size() << endl;    // 3
cout << s.top() << endl;    // 30
s.pop();
cout << s.top() << endl;    // 20
cout << s.empty() << endl;  // 0 (false)
```

Answer: 3 30 20 0

Explanation:

- Size: 3 elements
 - Top: 30 (last pushed)
 - After pop: top is 20
 - Empty: 0 (false, as 2 elements remain)
-

Question 18: Modifying Top Element

Question: Output of modifying top element?

```
cpp
s.push(10);
s.push(20);
s.push(30);
cout << s.top() << endl; // 30
s.top() = 40;           // Modify top
cout << s.top() << endl; // 40
```

Answer: 30 40

Explanation: `s.top()` returns a reference to the top element, allowing modification. After changing it to 40, the next `top()` call returns the modified value.

Question 19: Runtime Error in Loop

Question: Output of popping and printing in a loop?

```
cpp
s.push(10);
s.push(20);
s.push(30);
s.push(40);
while(!s.empty()){
    s.pop();
    cout << s.top() << " ";
}
```

Answer: The program will give a runtime error

Explanation: The loop pops first, then tries to access top. When the last element is popped, the stack becomes empty, and calling `top()` on an empty stack causes undefined behavior/runtime error.

Question 20: Reversing Stack Elements

Question: Output of transferring elements between two stacks?

```
cpp
```

```
stack<int> s1, s2;
s1.push(10);
s1.push(20);
s1.push(30);
while(!s1.empty()){
    s2.push(s1.top());
    s1.pop();
}
while(!s2.empty()){
    cout << s2.top() << " ";
    s2.pop();
}
```

Answer: 10 20 30

Explanation:

- s1: [10, 20, 30]
- Transfer to s2: [30, 20, 10]
- Print s2: 10, 20, 30 (reversed order)

This technique reverses the stack by using another stack.

Question 21: Push Pseudocode

Question: What does the pseudocode do?

```
if stack is full
    return
else
    increment top
    stack[top] assign value
```

Answer: Push an element to the stack

Explanation: This is the classic push operation pseudocode. It checks for overflow, then increments the top pointer and assigns the value at that position.

Question 22: Recursion Data Structure

Question: Which data structure is used in recursion?

Answer: Stack

Explanation: The system uses a call stack to manage recursive function calls. Each function call is pushed onto the stack with its local variables and return address, and popped when the function returns.

Question 23: Reverse Words with Count Limit

Question: Output with count = 5?

```
cpp

string str = "Hello World";
// Processing first 5 characters: H-e-l-l-o
// Reversing: o-l-l-e-h
```

Answer: olleH

Explanation: The code processes only the first 5 characters of "Hello World" (H-e-l-l-o), pushes them onto a stack, then pops them out in reverse order, producing "olleH".

Question 24: Reverse Words with Count = 9

Question: Output with count = 9?

```
cpp

string str = "Hello World";
// Processing first 9 characters: H-e-l-l-o-[space]-W-o-r
// "Hello" reversed = "olleH"
// Space encountered, prints "olleH "
// Remaining "Wor" reversed = "roW"
```

Answer: olleH roW

Explanation: Processes first 9 characters. When space is encountered after "Hello", it reverses and prints "olleH ", then continues with "Wor", reverses it to "roW".

Question 25: Tower of Hanoi

Question: Output of Tower of Hanoi with n=3?

Answer: Move disk 1 from rod S to rod D

Explanation: The code implements Tower of Hanoi iteratively. With 3 disks, it takes 7 moves (2^3-1). The flag parameter ensures only the last move is printed. The final move (7th move) transfers disk 1 from source (S) to

destination (D).

The algorithm:

- For odd-numbered disks, swap auxiliary and destination
 - Moves follow pattern: src-dest, src-aux, aux-dest, repeating
 - Only the last move ($i == \text{totalMoves}$) is displayed
-

Summary of Key Concepts

Time Complexities

- **Push:** $O(1)$
- **Pop:** $O(1)$
- **Top/Peak:** $O(1)$
- **Empty:** $O(1)$
- **Search:** $O(n)$

Important Stack Properties

1. **LIFO Order:** Last In, First Out
2. **Operations:** Only at the top
3. **Underflow:** Pop from empty stack
4. **Overflow:** Push to full stack
5. **Applications:** Recursion, expression evaluation, backtracking, histogram problems

C++ STL Stack Syntax

```
cpp
#include <stack>
stack<int> s;      // Declaration
s.push(value);    // Insert
s.pop();          // Remove
s.top();          // Access top
s.empty();        // Check if empty
s.size();         // Get size
```